



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

STUDENT NAME:B.Anjali

REGISTER NUMBER:192371062

COURSE CODE:CSA0903

COURSE NAME: Programming in java

GUIDED BY: Magesh kumar

DATE:17-07-2026

1. Single Inheritance – Scenario

Scenario:

A company has a Vehicle class with methods startEngine() and stopEngine(). A Car

class extends Vehicle and adds a playMusic() method.

The company now wants every car to automatically log the engine start time whenever

startEngine() is called, while still using the original engine-start functionality.

Questions:

1. How would you implement this using single inheritance?
2. Should Car override startEngine()? Why?
3. How can the parent class implementation still be executed?
4. What happens if Car does not override startEngine()? i need to write as assignment but in word give according to that

1. Single Inheritance – Scenario

Scenario

A company has a Vehicle class with methods startEngine() and stopEngine(). A Car class extends Vehicle and adds a playMusic()

method. The company wants every car to automatically log the engine start time whenever `startEngine()` is called while still using the original engine-start functionality.

Answer 1: How would you implement this using single inheritance?

Create a `Vehicle` class containing the methods `startEngine()` and `stopEngine()`. Then create a `Car` class that extends `Vehicle`. In the `Car` class, override the `startEngine()` method to add the engine start time logging feature. Call the parent class method using `super.startEngine()` so that the original engine-start functionality is executed before logging the start time.

Answer 2: Should Car override startEngine()? Why?

Yes. The `Car` class should override `startEngine()` because it needs to perform an additional task, which is logging the engine start time, while still keeping the original functionality of starting the engine.

Answer 3: How can the parent class implementation still be executed?

The parent class implementation can be executed by using the `super` keyword.

Example:

```
super.startEngine();
```

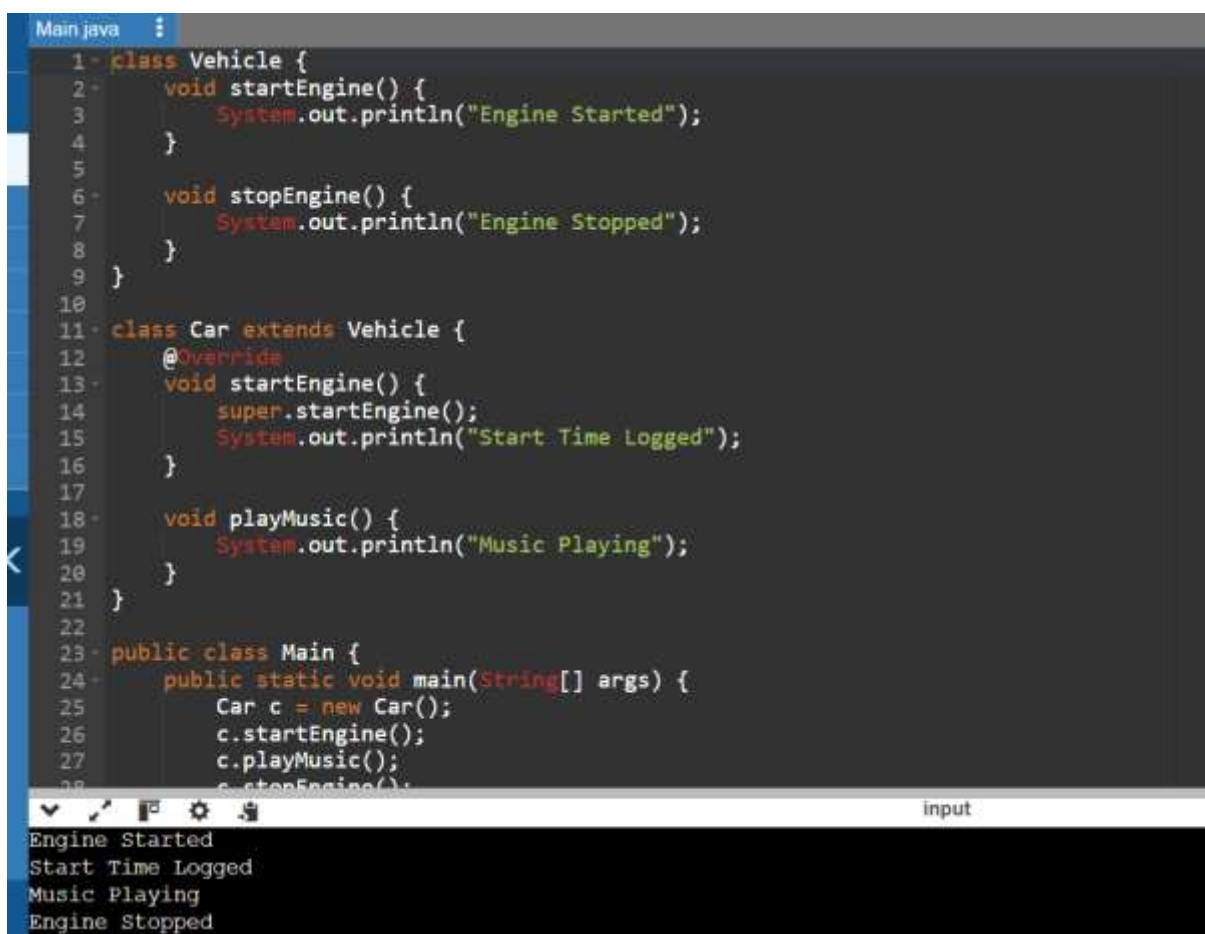
This calls the `startEngine()` method of the `Vehicle` class before executing the additional code in the `Car` class.

Answer 4: What happens if Car does not override startEngine()? If the `Car` class does not override `startEngine()`, it will automatically inherit the `Vehicle` class implementation. Only the original engine-start functionality will execute, and the engine start time will not be logged.

Sample Java Program:

```
class Vehicle {  
    void startEngine() {  
        System.out.println("Engine Started");  
    }  
  
    void stopEngine() {  
        System.out.println("Engine Stopped");  
    }  
}  
  
class Car extends Vehicle {  
    @Override  
    void startEngine() {  
        super.startEngine();  
        System.out.println("Engine Start Time Logged");  
    }  
  
    void playMusic() {  
        System.out.println("Music Playing");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {
```

```
    Car car = new Car();  
    car.startEngine();  
    car.playMusic();  
    car.stopEngine();  
}  
}
```



```
Main.java  
1 class Vehicle {  
2     void startEngine() {  
3         System.out.println("Engine Started");  
4     }  
5  
6     void stopEngine() {  
7         System.out.println("Engine Stopped");  
8     }  
9 }  
10  
11 class Car extends Vehicle {  
12     @Override  
13     void startEngine() {  
14         super.startEngine();  
15         System.out.println("Start Time Logged");  
16     }  
17  
18     void playMusic() {  
19         System.out.println("Music Playing");  
20     }  
21 }  
22  
23 public class Main {  
24     public static void main(String[] args) {  
25         Car c = new Car();  
26         c.startEngine();  
27         c.playMusic();  
28         c.stopEngine();  
29     }  
30 }  
31  
32  
33  
34  
35  
36  
37  
38  
39  
40  
41  
42  
43  
44  
45  
46  
47  
48  
49  
50  
51  
52  
53  
54  
55  
56  
57  
58  
59  
60  
61  
62  
63  
64  
65  
66  
67  
68  
69  
70  
71  
72  
73  
74  
75  
76  
77  
78  
79  
80  
81  
82  
83  
84  
85  
86  
87  
88  
89  
90  
91  
92  
93  
94  
95  
96  
97  
98  
99  
100  
101  
102  
103  
104  
105  
106  
107  
108  
109  
110  
111  
112  
113  
114  
115  
116  
117  
118  
119  
120  
121  
122  
123  
124  
125  
126  
127  
128  
129  
130  
131  
132  
133  
134  
135  
136  
137  
138  
139  
140  
141  
142  
143  
144  
145  
146  
147  
148  
149  
150  
151  
152  
153  
154  
155  
156  
157  
158  
159  
160  
161  
162  
163  
164  
165  
166  
167  
168  
169  
170  
171  
172  
173  
174  
175  
176  
177  
178  
179  
180  
181  
182  
183  
184  
185  
186  
187  
188  
189  
190  
191  
192  
193  
194  
195  
196  
197  
198  
199  
200  
201  
202  
203  
204  
205  
206  
207  
208  
209  
210  
211  
212  
213  
214  
215  
216  
217  
218  
219  
220  
221  
222  
223  
224  
225  
226  
227  
228  
229  
230  
231  
232  
233  
234  
235  
236  
237  
238  
239  
240  
241  
242  
243  
244  
245  
246  
247  
248  
249  
250  
251  
252  
253  
254  
255  
256  
257  
258  
259  
260  
261  
262  
263  
264  
265  
266  
267  
268  
269  
270  
271  
272  
273  
274  
275  
276  
277  
278  
279  
280  
281  
282  
283  
284  
285  
286  
287  
288  
289  
290  
291  
292  
293  
294  
295  
296  
297  
298  
299  
300  
301  
302  
303  
304  
305  
306  
307  
308  
309  
310  
311  
312  
313  
314  
315  
316  
317  
318  
319  
320  
321  
322  
323  
324  
325  
326  
327  
328  
329  
330  
331  
332  
333  
334  
335  
336  
337  
338  
339  
340  
341  
342  
343  
344  
345  
346  
347  
348  
349  
350  
351  
352  
353  
354  
355  
356  
357  
358  
359  
360  
361  
362  
363  
364  
365  
366  
367  
368  
369  
370  
371  
372  
373  
374  
375  
376  
377  
378  
379  
380  
381  
382  
383  
384  
385  
386  
387  
388  
389  
390  
391  
392  
393  
394  
395  
396  
397  
398  
399  
400  
401  
402  
403  
404  
405  
406  
407  
408  
409  
410  
411  
412  
413  
414  
415  
416  
417  
418  
419  
420  
421  
422  
423  
424  
425  
426  
427  
428  
429  
430  
431  
432  
433  
434  
435  
436  
437  
438  
439  
440  
441  
442  
443  
444  
445  
446  
447  
448  
449  
450  
451  
452  
453  
454  
455  
456  
457  
458  
459  
460  
461  
462  
463  
464  
465  
466  
467  
468  
469  
470  
471  
472  
473  
474  
475  
476  
477  
478  
479  
480  
481  
482  
483  
484  
485  
486  
487  
488  
489  
490  
491  
492  
493  
494  
495  
496  
497  
498  
499  
500  
501  
502  
503  
504  
505  
506  
507  
508  
509  
510  
511  
512  
513  
514  
515  
516  
517  
518  
519  
520  
521  
522  
523  
524  
525  
526  
527  
528  
529  
530  
531  
532  
533  
534  
535  
536  
537  
538  
539  
540  
541  
542  
543  
544  
545  
546  
547  
548  
549  
550  
551  
552  
553  
554  
555  
556  
557  
558  
559  
560  
561  
562  
563  
564  
565  
566  
567  
568  
569  
570  
571  
572  
573  
574  
575  
576  
577  
578  
579  
580  
581  
582  
583  
584  
585  
586  
587  
588  
589  
590  
591  
592  
593  
594  
595  
596  
597  
598  
599  
600  
601  
602  
603  
604  
605  
606  
607  
608  
609  
610  
611  
612  
613  
614  
615  
616  
617  
618  
619  
620  
621  
622  
623  
624  
625  
626  
627  
628  
629  
630  
631  
632  
633  
634  
635  
636  
637  
638  
639  
640  
641  
642  
643  
644  
645  
646  
647  
648  
649  
650  
651  
652  
653  
654  
655  
656  
657  
658  
659  
660  
661  
662  
663  
664  
665  
666  
667  
668  
669  
670  
671  
672  
673  
674  
675  
676  
677  
678  
679  
680  
681  
682  
683  
684  
685  
686  
687  
688  
689  
690  
691  
692  
693  
694  
695  
696  
697  
698  
699  
700  
701  
702  
703  
704  
705  
706  
707  
708  
709  
710  
711  
712  
713  
714  
715  
716  
717  
718  
719  
720  
721  
722  
723  
724  
725  
726  
727  
728  
729  
730  
731  
732  
733  
734  
735  
736  
737  
738  
739  
740  
741  
742  
743  
744  
745  
746  
747  
748  
749  
750  
751  
752  
753  
754  
755  
756  
757  
758  
759  
760  
761  
762  
763  
764  
765  
766  
767  
768  
769  
770  
771  
772  
773  
774  
775  
776  
777  
778  
779  
780  
781  
782  
783  
784  
785  
786  
787  
788  
789  
790  
791  
792  
793  
794  
795  
796  
797  
798  
799  
800  
801  
802  
803  
804  
805  
806  
807  
808  
809  
810  
811  
812  
813  
814  
815  
816  
817  
818  
819  
820  
821  
822  
823  
824  
825  
826  
827  
828  
829  
830  
831  
832  
833  
834  
835  
836  
837  
838  
839  
840  
841  
842  
843  
844  
845  
846  
847  
848  
849  
850  
851  
852  
853  
854  
855  
856  
857  
858  
859  
860  
861  
862  
863  
864  
865  
866  
867  
868  
869  
870  
871  
872  
873  
874  
875  
876  
877  
878  
879  
880  
881  
882  
883  
884  
885  
886  
887  
888  
889  
890  
891  
892  
893  
894  
895  
896  
897  
898  
899  
900  
901  
902  
903  
904  
905  
906  
907  
908  
909  
910  
911  
912  
913  
914  
915  
916  
917  
918  
919  
920  
921  
922  
923  
924  
925  
926  
927  
928  
929  
930  
931  
932  
933  
934  
935  
936  
937  
938  
939  
940  
941  
942  
943  
944  
945  
946  
947  
948  
949  
950  
951  
952  
953  
954  
955  
956  
957  
958  
959  
960  
961  
962  
963  
964  
965  
966  
967  
968  
969  
970  
971  
972  
973  
974  
975  
976  
977  
978  
979  
980  
981  
982  
983  
984  
985  
986  
987  
988  
989  
990  
991  
992  
993  
994  
995  
996  
997  
998  
999  
1000  
1001  
1002  
1003  
1004  
1005  
1006  
1007  
1008  
1009  
1010  
1011  
1012  
1013  
1014  
1015  
1016  
1017  
1018  
1019  
1020  
1021  
1022  
1023  
1024  
1025  
1026  
1027  
1028  
1029  
1030  
1031  
1032  
1033  
1034  
1035  
1036  
1037  
1038  
1039  
1040  
1041  
1042  
1043  
1044  
1045  
1046  
1047  
1048  
1049  
1050  
1051  
1052  
1053  
1054  
1055  
1056  
1057  
1058  
1059  
1060  
1061  
1062  
1063  
1064  
1065  
1066  
1067  
1068  
1069  
1070  
1071  
1072  
1073  
1074  
1075  
1076  
1077  
1078  
1079  
1080  
1081  
1082  
1083  
1084  
1085  
1086  
1087  
1088  
1089  
1090  
1091  
1092  
1093  
1094  
1095  
1096  
1097  
1098  
1099  
1100  
1101  
1102  
1103  
1104  
1105  
1106  
1107  
1108  
1109  
1110  
1111  
1112  
1113  
1114  
1115  
1116  
1117  
1118  
1119  
1120  
1121  
1122  
1123  
1124  
1125  
1126  
1127  
1128  
1129  
1130  
1131  
1132  
1133  
1134  
1135  
1136  
1137  
1138  
1139  
1140  
1141  
1142  
1143  
1144  
1145  
1146  
1147  
1148  
1149  
1150  
1151  
1152  
1153  
1154  
1155  
1156  
1157  
1158  
1159  
1160  
1161  
1162  
1163  
1164  
1165  
1166  
1167  
1168  
1169  
1170  
1171  
1172  
1173  
1174  
1175  
1176  
1177  
1178  
1179  
1180  
1181  
1182  
1183  
1184  
1185  
1186  
1187  
1188  
1189  
1190  
1191  
1192  
1193  
1194  
1195  
1196  
1197  
1198  
1199  
1200  
1201  
1202  
1203  
1204  
1205  
1206  
1207  
1208  
1209  
1210  
1211  
1212  
1213  
1214  
1215  
1216  
1217  
1218  
1219  
1220  
1221  
1222  
1223  
1224  
1225  
1226  
1227  
1228  
1229  
1230  
1231  
1232  
1233  
1234  
1235  
1236  
1237  
1238  
1239  
1240  
1241  
1242  
1243  
1244  
1245  
1246  
1247  
1248  
1249  
1250  
1251  
1252  
1253  
1254  
1255  
1256  
1257  
1258  
1259  
1260  
1261  
1262  
1263  
1264  
1265  
1266  
1267  
1268  
1269  
1270  
1271  
1272  
1273  
1274  
1275  
1276  
1277  
1278  
1279  
1280  
1281  
1282  
1283  
1284  
1285  
1286  
1287  
1288  
1289  
1290  
1291  
1292  
1293  
1294  
1295  
1296  
1297  
1298  
1299  
1300  
1301  
1302  
1303  
1304  
1305  
1306  
1307  
1308  
1309  
1310  
1311  
1312  
1313  
1314  
1315  
1316  
1317  
1318  
1319  
1320  
1321  
1322  
1323  
1324  
1325  
1326  
1327  
1328  
1329  
1330  
1331  
1332  
1333  
1334  
1335  
1336  
1337  
1338  
1339  
1340  
1341  
1342  
1343  
1344  
1345  
1346  
1347  
1348  
1349  
1350  
1351  
1352  
1353  
1354  
1355  
1356  
1357  
1358  
1359  
1360  
1361  
1362  
1363  
1364  
1365  
1366  
1367  
1368  
1369  
1370  
1371  
1372  
1373  
1374  
1375  
1376  
1377  
1378  
1379  
1380  
1381  
1382  
1383  
1384  
1385  
1386  
1387  
1388  
1389  
1390  
1391  
1392  
1393  
1394  
1395  
1396  
1397  
1398  
1399  
1400  
1401  
1402  
1403  
1404  
1405  
1406  
1407  
1408  
1409  
1410  
1411  
1412  
1413  
1414  
1415  
1416  
1417  
1418  
1419  
1420  
1421  
1422  
1423  
1424  
1425  
1426  
1427  
1428  
1429  
1430  
1431  
1432  
1433  
1434  
1435  
1436  
1437  
1438  
1439  
1440  
1441  
1442  
1443  
1444  
1445  
1446  
1447  
1448  
1449  
1450  
1451  
1452  
1453  
1454  
1455  
1456  
1457  
1458  
1459  
1460  
1461  
1462  
1463  
1464  
1465  
1466  
1467  
1468  
1469  
1470  
1471  
1472  
1473  
1474  
1475  
1476  
1477  
1478  
1479  
1480  
1481  
1482  
1483  
1484  
1485  
1486  
1487  
1488  
1489  
1490  
1491  
1492  
1493  
1494  
1495  
1496  
1497  
1498  
1499  
1500  
1501  
1502  
1503  
1504  
1505  
1506  
1507  
1508  
1509  
1510  
1511  
1512  
1513  
1514  
1515  
1516  
1517  
1518  
1519  
1520  
1521  
1522  
1523  
1524  
1525  
1526  
1527  
1528  
1529  
1530  
1531  
1532  
1533  
1534  
1535  
1536  
1537  
1538  
1539  
1540  
1541  
1542  
1543  
1544  
1545  
1546  
1547  
1548  
1549  
1550  
1551  
1552  
1553  
1554  
1555  
1556  
1557  
1558  
1559  
1560  
1561  
1562  
1563  
1564  
1565  
1566  
1567  
1568  
1569  
1570  
1571  
1572  
1573  
1574  
1575  
1576  
1577  
1578  
1579  
1580  
1581  
1582  
1583  
1584  
1585  
1586  
1587  
1588  
1589  
1590  
1591  
1592  
1593  
1594  
1595  
1596  
1597  
1598  
1599  
1600  
1601  
1602  
1603  
1604  
1605  
1606  
1607  
1608  
1609  
1610  
1611  
1612  
1613  
1614  
1615  
1616  
1617  
1618  
1619  
1620  
1621  
1622  
1623  
1624  
1625  
1626  
1627  
1628  
1629  
1630  
1631  
1632  
1633  
1634  
1635  
1636  
1637  
1638  
1639  
1640  
1641  
1642  
1643  
1644  
1645  
1646  
1647  
1648  
1649  
1650  
1651  
1652  
1653  
1654  
1655  
1656  
1657  
1658  
1659  
1660  
1661  
1662  
1663  
1664  
1665  
1666  
1667  
1668  
1669  
1670  
1671  
1672  
1673  
1674  
1675  
1676  
1677  
1678  
1679  
1680  
1681  
1682  
1683  
1684  
1685  
1686  
1687  
1688  
1689  
1690  
1691  
1692  
1693  
1694  
1695  
1696  
1697  
1698  
1699  
1700  
1701  
1702  
1703  
1704  
1705  
1706  
1707  
1708  
1709  
1710  
1711  
1712  
1713  
1714  
1715  
1716  
1717  
1718  
1719  
1720  
1721  
1722  
1723  
1724  
1725  
1726  
1727  
1728  
1729  
1730  
1731  
1732  
1733  
1734  
1735  
1736  
1737  
1738  
1739  
1740  
1741  
1742  
1743  
1744  
1745  
1746  
1747  
1748  
1749  
1750  
1751  
1752  
1753  
1754  
1755  
1756  
1757  
1758  
1759  
1760  
1761  
1762  
1763  
1764  
1765  
1766  
1767  
1768  
1769  
1770  
1771  
1772  
1773  
1774  
1775  
1776  
1777  
1778  
1779  
1780  
1781  
1782  
1783  
1784  
1785  
1786  
1787  
1788  
1789  
1790  
1791  
1792  
1793  
1794  
1795  
1796  
1797  
1798  
1799  
1800  
1801  
1802  
1803  
1804  
1805  
1806  
1807  
1808  
1809  
1810  
1811  
1812  
1813  
1814  
1815  
1816  
1817  
1818  
1819  
1820  
1821  
1822  
1823  
1824  
1825  
1826  
1827  
1828  
1829  
1830  
1831  
1832  
1833  
1834  
1835  
1836  
1837  
1838  
1839  
1840  
1841  
1842  
1843  
1844  
1845  
1846  
1847  
1848  
1849  
1850  
1851  
1852  
1853  
1854  
1855  
1856  
1857  
1858  
1859  
1860  
1861  
1862  
1863  
1864  
1865  
1866  
1867  
1868  
1869  
1870  
1871  
1872  
1873  
1874  
1875  
1876  
1877  
1878  
1879  
1880  
1881  
1882  
1883  
1884  
1885  
1886  
1887  
1888  
1889  
1890  
1891  
1892  
1893  
1894  
1895  
1896  
1897  
1898  
1899  
1900  
1901  
1902  
1903  
1904  
1905  
1906  
1907  
1908  
1909  
1910  
1911  
1912  
1913  
1914  
1915  
1916  
1917  
1918  
1919  
1920  
1921  
1922  
1923  
1924  
1925  
1926  
1927  
1928  
1929  
1930  
1931  
1932  
1933  
1934  
1935  
1936  
1937  
1938  
1939  
1940  
1941  
1942  
1943  
1944  
1945  
1946  
1947  
1948  
1949  
1950  
1951  
1952  
1953  
1954  
1955  
1956  
1957  
1958  
1959  
1960  
1961  
1962  
1963  
1964  
1965  
1966  
1967  
1968  
1969  
1970  
1971  
1972  
1973  
1974  
1975  
1976  
1977  
1978  
1979  
1980  
1981  
1982  
1983  
1984  
1985  
1986  
1987  
1988  
1989  
1990  
1991  
1992  
1993  
1994  
1995  
1996  
1997  
1998  
1999  
2000  
2001  
2002  
2003  
2004  
2005  
2006  
2007  
2008  
2009  
2010  
2011  
2012  
2013  
2014  
2015  
2016  
2017  
2018  
2019  
2020  
2021  
2022  
2023  
2024  
2025  
2026  
2027  
2028  
2029  
2030  
2031  
2032  
2033  
2034  
2035  
2036  
2037  
2038  
2039  
2040  
2041  
2042  
2043  
2044  
2045  
2046  
2047  
2048  
2049  
2050  
2051  
2052  
2053  
2054  
2055  
2056  
2057  
2058  
2059  
2060  
2061  
2062  
2063  
2064  
2065  
2066  
2067  
2068  
2069  
2070  
2071  
2072  
2073  
2074  
2075  
2076  
2077  
2078  
2079  
2080  
2081  
2082  
2083  
2084  
2085  
2086  
2087  
2088  
2089  
2090  
2091  
2092  
2093  
2094  
2095  
2096  
2097  
2098  
2099  
2100  
2101  
2102  
2103  
2104  
2105  
2106  
2107  
2108  
2109  
2110  
2111  
2112  
2113  
2114  
2115  
2116  
2117  
2118  
2119  
2120  
2121  
2122  
2123  
2124  
2125  
2126  
2127  
2128  
2129  
2130  
2131  
2132  
2133  
2134  
2135  
2136  
2137  
2138  
2139  
2140  
2141  
2142  
2143  
2144  
2145  
2146  
2147  
2148  
2149  
2150  
2151  
2152  
2153  
2154  
2155  
2156  
2157  
2158  
2159  
2160  
2161  
2162  
2163  
2164  
2165  
2166  
2167  
2168  
2169  
2170  
2171  
2172  
2173  
2174  
2175  
2176  
2177  
2178  
2179  
2180  
2181  
2182  
2183  
2184  
2185  
2186  
2187  
2188  
2189  
2190  
2191  
2192  
2193  
2194  
2195  
2196  
2197  
2198  
2199
```

2. Multilevel Inheritance – Scenario

Scenario:

A software company models employees as follows:

- Employee → stores employee ID and name.
- Developer extends Employee → stores programming language.
- SeniorDeveloper extends Developer → stores team size and performs code reviews.

Now HR requests that every SeniorDeveloper should be able to access employee information and programming language details without duplicating code.

Questions:

1. Why is multilevel inheritance suitable here?
2. Which members are accessible in SeniorDeveloper?
3. If Developer overrides a method from Employee, which implementation will SeniorDeveloper inherit?
4. Draw the inheritance hierarchy.

Multilevel Inheritance – Scenario

Scenario:

A software company models employees using three classes. The Employee class stores the employee ID and name. The Developer class extends Employee and stores the programming language. The SeniorDeveloper class extends Developer and stores the team size and performs code reviews. HR wants every SeniorDeveloper to access employee information and programming language details without duplicating code.

Answer 1: Why is multilevel inheritance suitable here?

Multilevel inheritance is suitable because each class builds on the previous one. The Developer class inherits the employee details from the Employee class, and the SeniorDeveloper class inherits both employee details and programming language from the Developer class. This avoids code duplication and improves code reusability.

Answer 2: Which members are accessible in SeniorDeveloper?

The SeniorDeveloper class can access:

- Employee ID
- Employee Name
- Programming Language
- Team Size
- Employee class methods
- Developer class methods
- SeniorDeveloper class methods

Thus, SeniorDeveloper inherits all accessible members from both Employee and Developer.

Answer 3: If Developer overrides a method from Employee, which implementation will SeniorDeveloper inherit?

If the Developer class overrides a method from the Employee class, then the SeniorDeveloper class inherits the overridden method from the Developer class. The most recently overridden version is executed unless SeniorDeveloper overrides it again.

Answer 4: Draw the inheritance hierarchy.

Employee

|

Developer

|

SeniorDeveloper

Sample Java Program:

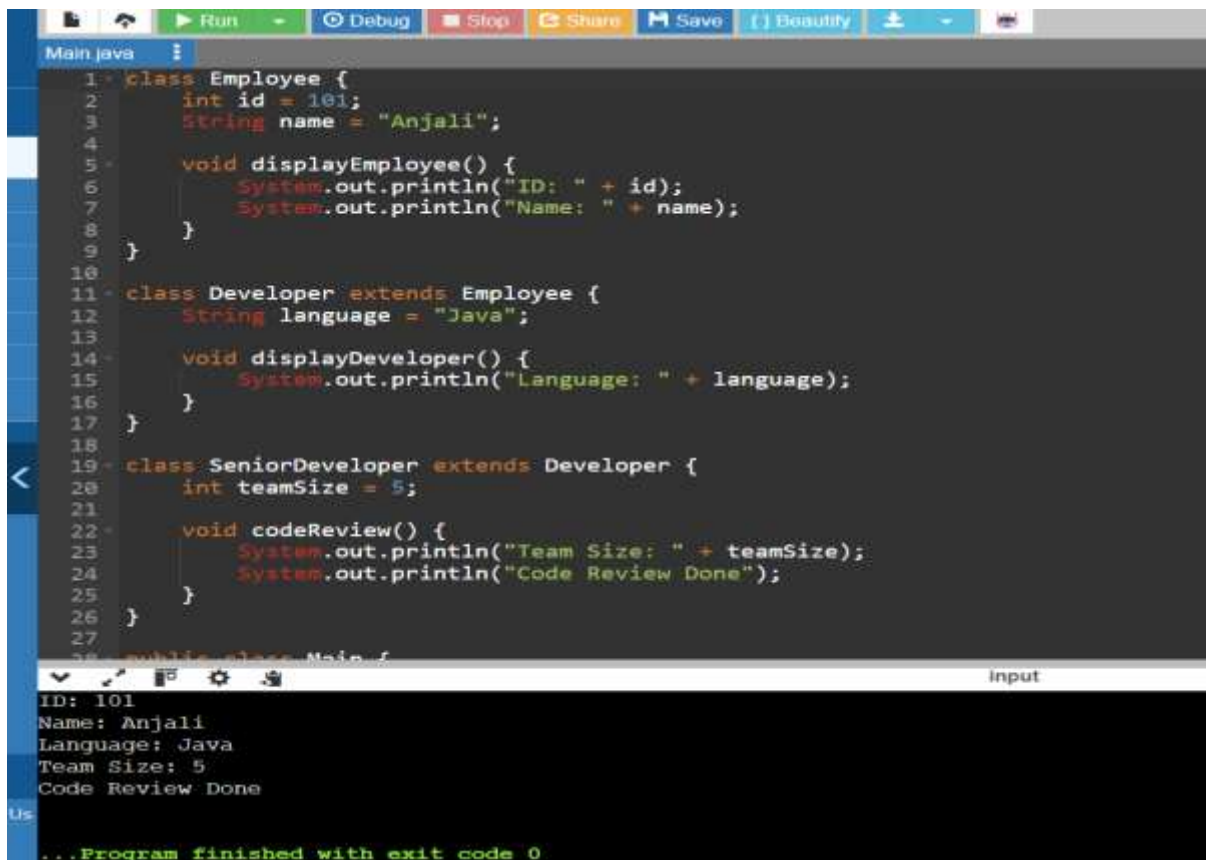
```
class Employee {  
    int id = 101;  
    String name = "Anjali";  
  
    void displayEmployee() {  
        System.out.println("ID: " + id);  
        System.out.println("Name: " + name);  
    }  
}  
  
class Developer extends Employee {  
    String language = "Java";  
    void displayDeveloper() {  
        System.out.println("Programming Language: " + language);  
    }  
}  
  
class SeniorDeveloper extends Developer {  
    int teamSize = 5;  
    void codeReview() {
```

```

        System.out.println("Team Size: " + teamSize);
        System.out.println("Code Review Completed");
    }
}

public class Main {
    public static void main(String[] args) {
        SeniorDeveloper sd = new SeniorDeveloper();
        sd.displayEmployee();
        sd.displayDeveloper();
        sd.codeReview();
    }
}

```



The screenshot shows an IDE window titled 'Main.java' with the following Java code:

```

1 class Employee {
2     int id = 101;
3     String name = "Anjali";
4
5     void displayEmployee() {
6         System.out.println("ID: " + id);
7         System.out.println("Name: " + name);
8     }
9 }
10
11 class Developer extends Employee {
12     String language = "Java";
13
14     void displayDeveloper() {
15         System.out.println("Language: " + language);
16     }
17 }
18
19 class SeniorDeveloper extends Developer {
20     int teamSize = 5;
21
22     void codeReview() {
23         System.out.println("Team Size: " + teamSize);
24         System.out.println("Code Review Done");
25     }
26 }
27
28 public class Main {

```

The output window at the bottom shows the following text:

```

ID: 101
Name: Anjali
Language: Java
Team Size: 5
Code Review Done
...Program finished with exit code 0

```


3.Hierarchical Inheritance – Scenario

Scenario:

An online shopping application has a base class Payment.

Three classes inherit from it:

- ☐ CreditCardPayment
- ☐ UPIPayment
- ☐ NetBankingPayment

Each payment type processes transactions differently but shares common validation logic.

Questions:

1. Why is hierarchical inheritance appropriate here?
2. Where should common validation logic be implemented?
3. How can each payment class provide its own transaction processing?
4. What is the advantage over writing three independent classes?

.Hierarchical Inheritance – Scenario

Scenario

An online shopping application has a base class, Payment. Three classes—CreditCardPayment,upiPayment, and NetBankingPayment—inherit from it. Each payment type processes transactions differently but shares the same payment validation logic.

Answer 1: Why is hierarchical inheritance appropriate here?

Hierarchical inheritance is appropriate because multiple classes share common functionality from a single parent class. The Payment class contains common features, while each child class implements its own payment processing. This promotes code reuse and reduces duplication.

Answer 2: Where should common validation logic be implemented?

The common validation logic should be implemented in the **Payment** class. Since all payment methods require validation, placing it in the parent class allows every child class to inherit and use the same validation method.

Answer 3: How can each payment class provide its own transaction processing?

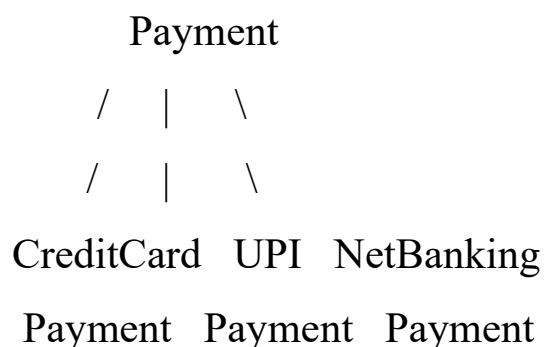
Each child class can define or override a `processPayment()` method with its own implementation. This allows `CreditCardPayment`, `UPIPayment`, and `NetBankingPayment` to process transactions according to their specific requirements.

Answer 4: What is the advantage over writing three independent classes?

Using hierarchical inheritance provides the following advantages:

- Avoids code duplication.
- Improves code reusability.
- Makes the program easier to maintain.
- Allows all payment classes to share common validation logic.
- Reduces development time and improves code organization.

Inheritance Hierarchy



Sample Java Program

```
class Payment {  
    void validate() {  
        System.out.println("Payment Validated");  
    }  
}  
  
class CreditCardPayment extends Payment {  
    void processPayment() {  
        System.out.println("Credit Card Payment Processed");  
    }  
}  
  
class UPIPayment extends Payment {  
    void processPayment() {  
        System.out.println("UPI Payment Processed");  
    }  
}  
  
class NetBankingPayment extends Payment {  
    void processPayment() {  
        System.out.println("Net Banking Payment Processed");  
    }  
}  
  
public class Main {
```

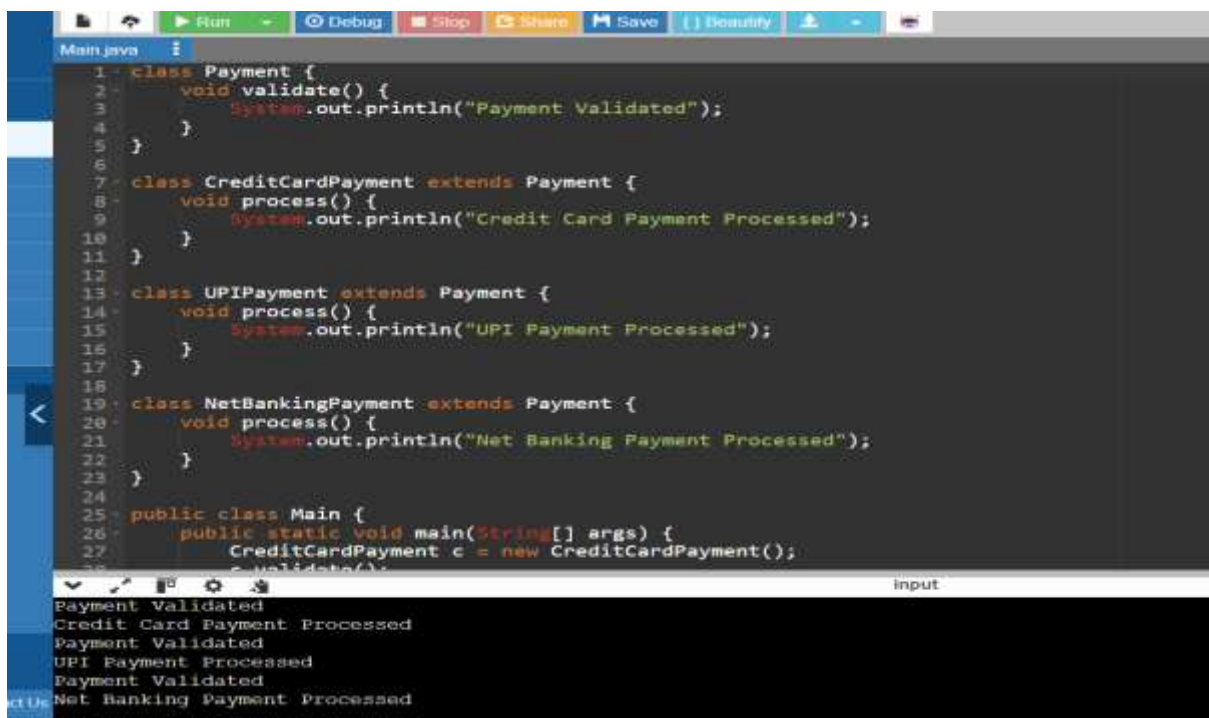
```

public static void main(String[] args) {
    CreditCardPayment c = new CreditCardPayment();
    c.validate();
    c.processPayment();

    UPIPayment u = new UPIPayment();
    u.validate();
    u.processPayment();

    NetBankingPayment n = new NetBankingPayment();
    n.validate();
    n.processPayment();
}
}

```



The screenshot shows an IDE window titled 'Main.java' with the following Java code:

```

1 class Payment {
2     void validate() {
3         System.out.println("Payment Validated");
4     }
5 }
6
7 class CreditCardPayment extends Payment {
8     void process() {
9         System.out.println("Credit Card Payment Processed");
10    }
11 }
12
13 class UPIPayment extends Payment {
14     void process() {
15         System.out.println("UPI Payment Processed");
16     }
17 }
18
19 class NetBankingPayment extends Payment {
20     void process() {
21         System.out.println("Net Banking Payment Processed");
22     }
23 }
24
25 public class Main {
26     public static void main(String[] args) {
27         CreditCardPayment c = new CreditCardPayment();
28         c.validate();
29         c.processPayment();
30
31         UPIPayment u = new UPIPayment();
32         u.validate();
33         u.processPayment();
34
35         NetBankingPayment n = new NetBankingPayment();
36         n.validate();
37         n.processPayment();
38     }
39 }

```

The output window at the bottom shows the following lines of text:

```

Payment Validated
Credit Card Payment Processed
Payment Validated
UPI Payment Processed
Payment Validated
Net Banking Payment Processed

```

4. Compile-Time Polymorphism (Method Overloading) –

Scenario

A calculator application supports addition of:

- ☐ Two integers
- ☐ Three integers
- ☐ Two decimal numbers

The product owner wants users to call the same method name (add) regardless of the input.

Questions:

1. Which OOP concept should be used?
 2. How will Java decide which method to invoke?
 3. Can methods be overloaded by changing only the return type?
 4. What happens if `add(5, 10.0)` is called?
- ### **4. Compile-Time Polymorphism (Method Overloading) – Scenario**

Scenario:

A calculator application supports addition of two integers, three integers, and two decimal numbers. The product owner wants users to use the same method name `add()` for all types of addition, regardless of the input.

Answer 1: Which OOP concept should be used?

The appropriate OOP concept is Compile-Time Polymorphism, also known as Method Overloading.

In method overloading, multiple methods have the same name but different parameter lists (different number or types of parameters). Java determines which method to execute during compilation.

Answer 2: How will Java decide which method to invoke?

Java selects the appropriate overloaded method based on the number, type, and order of the arguments passed to the method. This decision is made at compile time.

For example:

- `add(10, 20) → Calls add(int, int)`
- `add(10, 20, 30) → Calls add(int, int, int)`
- `add(10.5, 20.5) → Calls add(double, double)`

Answer 3: Can methods be overloaded by changing only the return type?

No.

Methods cannot be overloaded by changing only the return type. The parameter list must be different. If only the return type changes, Java will produce a compile-time error.

Answer 4: What happens if `add(5, 10.0)` is called?

The first argument is an **integer** and the second argument is a **double**. Java automatically promotes the integer value to a double and calls the method:

`add(double, double)`

The result will be:

15.0

Sample Java Program

```
class Calculator {  
    int add(int a, int b) {  
        return a + b; }  
}
```

```

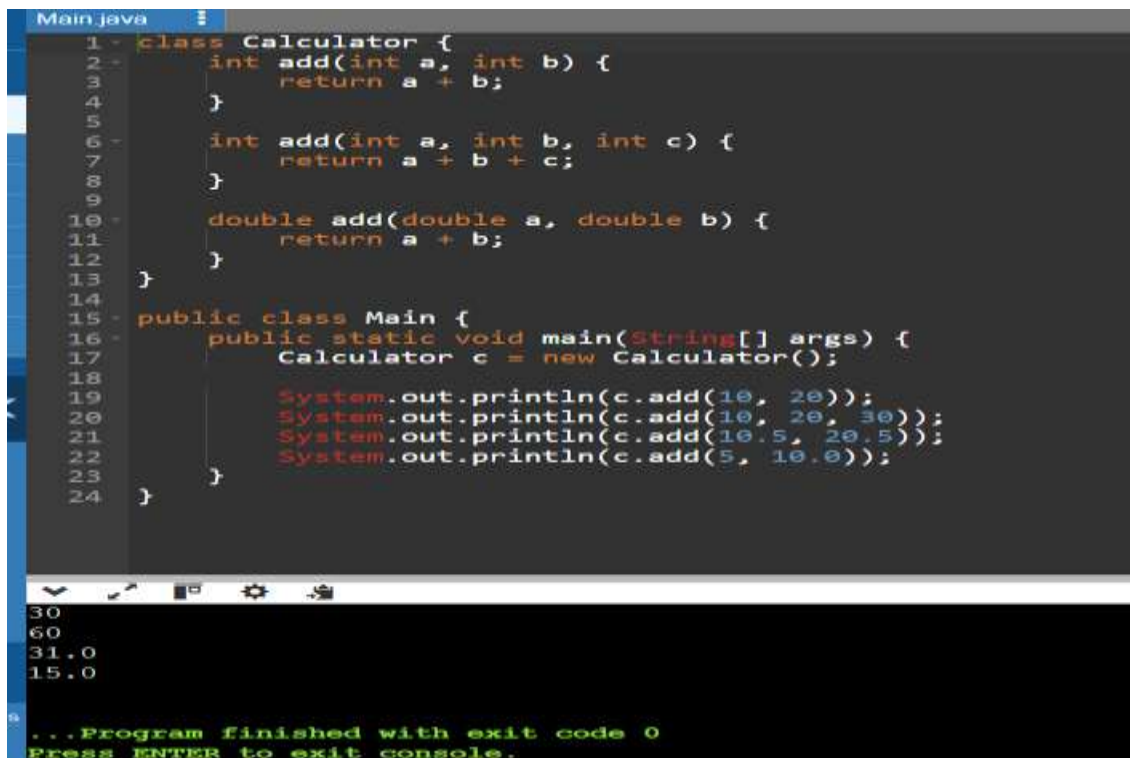
int add(int a, int b, int c) {
    return a + b + c;
}

double add(double a, double b) {
    return a + b;}

}

public class Main {
    public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println("Two Integers: " + c.add(10, 20));
        System.out.println("Three Integers: " + c.add(10, 20, 30));
        System.out.println("Two Decimals: " + c.add(10.5, 20.5));
        System.out.println("Mixed Values: " + c.add(5, 10.0));
    }
}

```



The screenshot shows a Java IDE with a file named 'Main.java'. The code is as follows:

```

1  class Calculator {
2      int add(int a, int b) {
3          return a + b;
4      }
5
6      int add(int a, int b, int c) {
7          return a + b + c;
8      }
9
10     double add(double a, double b) {
11         return a + b;
12     }
13 }
14
15 public class Main {
16     public static void main(String[] args) {
17         Calculator c = new Calculator();
18
19         System.out.println(c.add(10, 20));
20         System.out.println(c.add(10, 20, 30));
21         System.out.println(c.add(10.5, 20.5));
22         System.out.println(c.add(5, 10.0));
23     }
24 }

```

The output of the program is displayed in the console:

```

30
60
31.0
15.0

```

Below the output, the console shows the message: "...Program finished with exit code 0 Press ENTER to exit console."

5. Runtime Polymorphism (Method Overriding) –

Scenario

A food delivery application has:

Restaurant:

Pizza Burger Biryani

Each restaurant calculates delivery time differently.

The application stores all restaurants in:

List<Restaurant> restaurants;

and calls:

restaurant.calculateDeliveryTime();

without checking the actual restaurant type.

Questions:

1. Which OOP concept is being used?
2. Which version of calculateDeliveryTime() executes?
3. When is the method selection decided?
4. What would happen if calculateDeliveryTime() were declared static?

Runtime Polymorphism (Method Overriding) – Scenario

A food delivery application has a base class Restaurant. Three classes—Pizza, Burger, and Biryani—inherit from Restaurant. Each restaurant calculates the delivery time differently. The application stores all restaurant objects in a List<Restaurant> and calls restaurant.calculateDeliveryTime() without checking the actual restaurant type.

Answer 1: Which OOP concept is being used?

The OOP concept used is Runtime Polymorphism, also known as Method Overriding. In runtime polymorphism, a child class overrides a

method of the parent class, and the appropriate method is selected based on the actual object at runtime.

Answer 2: Which version of calculateDeliveryTime() executes?

The version of calculateDeliveryTime() that belongs to the actual object is executed.

For example:

- If the object is Pizza, then Pizza.calculateDeliveryTime() is executed.
- If the object is Burger, then Burger.calculateDeliveryTime() is executed.
- If the object is Biryani, then Biryani.calculateDeliveryTime() is executed.

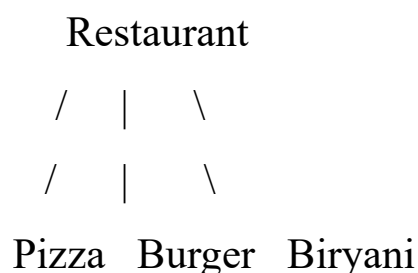
Answer 3: When is the method selection decided?

The method selection is decided at runtime (during program execution). Java checks the actual object type and calls the corresponding overridden method.

Answer 4: What would happen if calculateDeliveryTime() were declared static?

If calculateDeliveryTime() were declared static, it cannot be overridden. Instead, it would be hidden by the child classes. The method that executes would depend on the reference type, not the actual object type. Therefore, runtime polymorphism would not occur.

Class Hierarchy



Sample Java Program

```
class Restaurant {  
    void calculateDeliveryTime() {  
        System.out.println("General Delivery Time");  
    }  
}  
  
class Pizza extends Restaurant {  
    void calculateDeliveryTime() {  
        System.out.println("Pizza Delivery Time: 30 Minutes");  
    }  
}  
  
class Burger extends Restaurant {  
    void calculateDeliveryTime() {  
        System.out.println("Burger Delivery Time: 20 Minutes");  
    }  
}  
  
class Biryani extends Restaurant {  
    void calculateDeliveryTime() {  
        System.out.println("Biryani Delivery Time: 45 Minutes");  
    }  
}
```

```

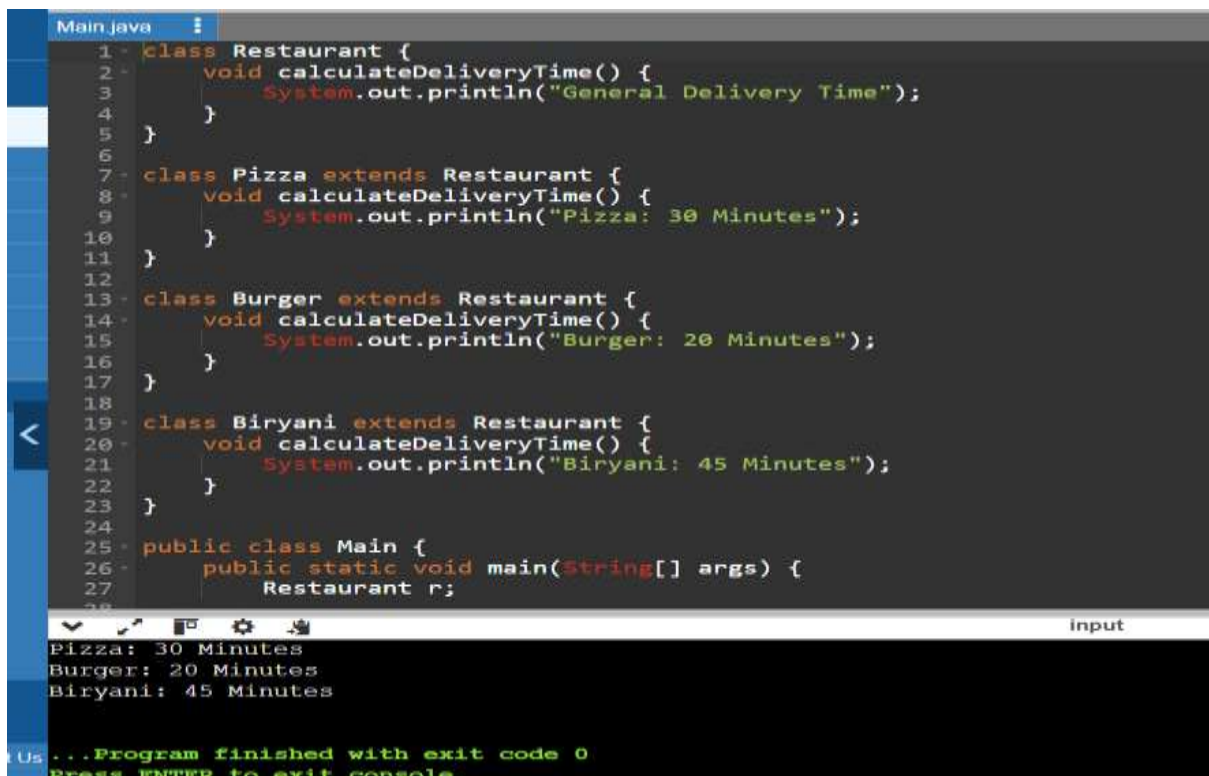
public class Main {
    public static void main(String[] args) {
        Restaurant r;

        r = new Pizza();
        r.calculateDeliveryTime();

        r = new Burger();
        r.calculateDeliveryTime();

        r = new Biryani();
        r.calculateDeliveryTime();
    }
}

```



The screenshot shows a Java IDE with a file named 'Main.java'. The code defines a base class 'Restaurant' with a 'calculateDeliveryTime()' method that prints 'General Delivery Time'. Three subclasses, 'Pizza', 'Burger', and 'Biryani', extend 'Restaurant' and override 'calculateDeliveryTime()' to print their respective delivery times: 'Pizza: 30 Minutes', 'Burger: 20 Minutes', and 'Biryani: 45 Minutes'. The 'Main' class contains a 'main' method that instantiates objects of each subclass and calls their 'calculateDeliveryTime()' methods.

```

1  class Restaurant {
2      void calculateDeliveryTime() {
3          System.out.println("General Delivery Time");
4      }
5  }
6
7  class Pizza extends Restaurant {
8      void calculateDeliveryTime() {
9          System.out.println("Pizza: 30 Minutes");
10     }
11 }
12
13 class Burger extends Restaurant {
14     void calculateDeliveryTime() {
15         System.out.println("Burger: 20 Minutes");
16     }
17 }
18
19 class Biryani extends Restaurant {
20     void calculateDeliveryTime() {
21         System.out.println("Biryani: 45 Minutes");
22     }
23 }
24
25 public class Main {
26     public static void main(String[] args) {
27         Restaurant r;
28
29         r = new Pizza();
30         r.calculateDeliveryTime();
31
32         r = new Burger();
33         r.calculateDeliveryTime();
34
35         r = new Biryani();
36         r.calculateDeliveryTime();
37     }
38 }

```

The output of the program is displayed in the console:

```

Pizza: 30 Minutes
Burger: 20 Minutes
Biryani: 45 Minutes
...Program finished with exit code 0
Press ENTER to exit console.

```

6. Interface – Scenario

Scenario:

A smart home application supports different devices:

- ☐ Smart Bulb
- ☐ Smart Fan
- ☐ Smart AC

All devices can be turned on and turned off, but each device has its own implementation.

In the future, third-party manufacturers should also be able to integrate their own devices

without modifying existing code.

Questions:

1. Would you use an interface or inheritance? Why?
2. Design an appropriate interface.
3. Can one device implement multiple interfaces? Give an example.
4. Why is an interface better than creating a common base class in this scenario?

6. Interface – Scenario

Scenario

A smart home application supports different devices such as Smart Bulb, Smart Fan, and Smart AC. All devices can be turned on and turned off, but each device has its own implementation. In the future, third-party manufacturers should also be able to add their own devices without modifying the existing code.

Answer 1: Would you use an interface or inheritance? Why?

An interface should be used because all smart devices provide the same basic operations (turnOn() and turnOff()), but each device has a

different implementation. Interfaces allow different classes to implement the same methods in their own way and make it easy to add new devices without changing existing code.

Answer 2: Design an appropriate interface.

The interface can be designed as follows:

```
interface SmartDevice {  
    void turnOn();  
    void turnOff();  
}
```

Any smart device such as SmartBulb, SmartFan, or SmartAC can implement this interface and provide its own implementation of these methods.

Answer 3: Can one device implement multiple interfaces? Give an example.

Yes. A class in Java can implement multiple interfaces.

Example:

```
interface SmartDevice {  
    void turnOn();  
    void turnOff();  
}
```

```
interface VoiceControl {  
    void voiceCommand();  
}
```

```
class SmartAC implements SmartDevice, VoiceControl {
```

```
public void turnOn() {  
    System.out.println("AC ON");  
}
```

```
public void turnOff() {  
    System.out.println("AC OFF");  
}
```

```
public void voiceCommand() {  
    System.out.println("Voice Command Enabled");  
}  
}
```

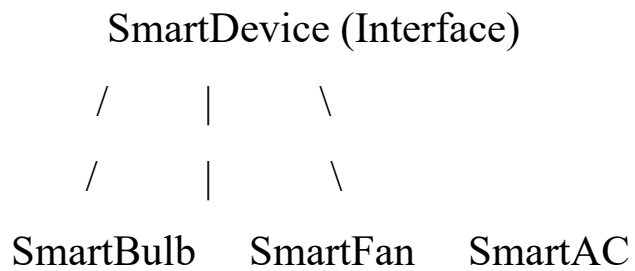
Here, the SmartAC class implements both SmartDevice and VoiceControl interfaces.

Answer 4: Why is an interface better than creating a common base class in this scenario?

An interface is better because:

- It allows different manufacturers to create their own smart devices without modifying existing code.
- It supports multiple interface implementation, which is not possible with class inheritance.
- It promotes flexibility and scalability.
- It ensures that every smart device provides the required methods (turnOn() and turnOff()) while allowing different implementations.
- It improves code reusability and maintainability.

Class Diagram



Sample Java Program:

```
interface SmartDevice {  
    void turnOn();  
    void turnOff();  
}
```

```
class SmartBulb implements SmartDevice {  
    public void turnOn() {  
        System.out.println("Smart Bulb ON");  
    }  
  
    public void turnOff() {  
        System.out.println("Smart Bulb OFF");  
    }  
}
```

```
class SmartFan implements SmartDevice {  
    public void turnOn() {  
        System.out.println("Smart Fan ON");  
    }  
}
```

```
}
```

```
public void turnOff() {  
    System.out.println("Smart Fan OFF");  
}
```

```
}
```

```
class SmartAC implements SmartDevice {  
    public void turnOn() {  
        System.out.println("Smart AC ON");  
    }
```

```
    public void turnOff() {  
        System.out.println("Smart AC OFF");  
    }
```

```
}
```

```
public class Main {  
    public static void main(String[] args) {  
        SmartDevice bulb = new SmartBulb();  
        SmartDevice fan = new SmartFan();  
        SmartDevice ac = new SmartAC();  
  
        bulb.turnOn();  
        bulb.turnOff();
```


fan.turnOn();

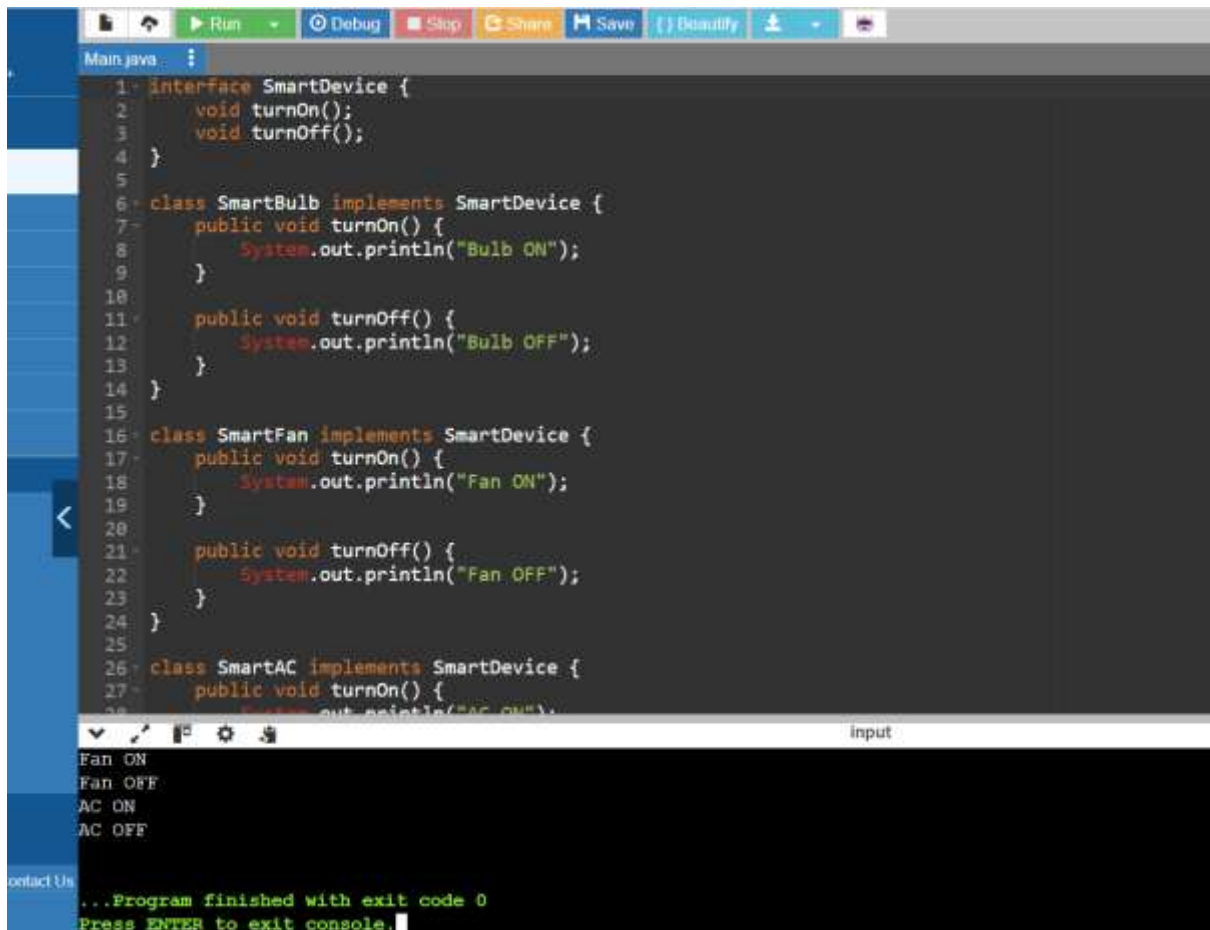
fan.turnOff();

ac.turnOn();

ac.turnOff();

}

}



```
1 interface SmartDevice {
2     void turnOn();
3     void turnOff();
4 }
5
6 class SmartBulb implements SmartDevice {
7     public void turnOn() {
8         System.out.println("Bulb ON");
9     }
10
11     public void turnOff() {
12         System.out.println("Bulb OFF");
13     }
14 }
15
16 class SmartFan implements SmartDevice {
17     public void turnOn() {
18         System.out.println("Fan ON");
19     }
20
21     public void turnOff() {
22         System.out.println("Fan OFF");
23     }
24 }
25
26 class SmartAC implements SmartDevice {
27     public void turnOn() {
28         System.out.println("AC ON");
29     }
30
31     public void turnOff() {
32         System.out.println("AC OFF");
33     }
34 }
```

input

Fan ON
Fan OFF
AC ON
AC OFF

...Program finished with exit code 0
Press ENTER to exit console.